

Intelligent System for Automatic Correction and Completion for Short Text Exchange

Project Description & Development Guidelines

1. Project Overview

This project aims to develop an intelligent NLP-powered system capable of automatically detecting and correcting errors in short text messages, as well as predicting and completing partial or ambiguous text in real-time conversational contexts. The system is designed to enhance communication quality in messaging platforms, chatbots, customer support tools, and any environment that relies on short-form text exchange.

By combining machine learning, deep learning, and linguistic models, the system provides context-aware suggestions that go beyond simple spell-checking — understanding intent, grammar, style, and semantic coherence.

2. Objectives

Automatic Error Correction: Detect and fix spelling, grammatical, and typographical errors in short text messages without user intervention.

Text Completion: Predict the most likely continuation of an incomplete sentence or phrase, offering meaningful suggestions based on context.

Contextual Awareness: Leverage conversational history and message context to improve the relevance of corrections and completions.

Low Latency: Ensure real-time performance suitable for live messaging environments, with response times under 200ms.

Multilingual Support: Design the architecture to support multiple languages, beginning with English and expanding iteratively.

User Adaptability: Allow the system to learn from user feedback and adapt corrections and completions to individual writing styles over time.

3. System Architecture

The system is composed of the following core modules:

3.1 Input Preprocessing Module

Responsible for tokenizing raw input text, normalizing characters, handling contractions, stripping unwanted symbols, and preparing text for downstream model consumption. This module also detects the language of the input text.

3.2 Error Detection & Correction Engine

Uses a fine-tuned transformer-based model (e.g., T5, BERT-based GEC models) trained on grammatical error correction datasets. The engine identifies spelling mistakes, wrong word choices, incorrect verb tenses, punctuation errors, and syntactic inconsistencies, then proposes corrected alternatives ranked by confidence score.

3.3 Text Completion Module

Leverages a generative language model (e.g., GPT-style architecture) fine-tuned on short conversational datasets. Given a partial input, the module generates N candidate completions, filtered and ranked using beam search and perplexity scoring.

3.4 Context Manager

Maintains a rolling window of recent conversation turns to provide the correction and completion modules with conversational context, improving the semantic relevance of outputs.

3.5 Feedback & Personalization Layer

Captures implicit feedback (accepted vs. rejected suggestions) and explicit ratings. Stores user preferences and vocabulary patterns to fine-tune suggestions over time using lightweight online learning techniques.

3.6 API Gateway

Exposes a RESTful (and optionally WebSocket) API for client integration. Handles authentication, rate limiting, request routing, and response formatting.

4. Recommended Technology Stack

Layer	Technology
Programming Language	Python 3.10+
NLP / ML Framework	Hugging Face Transformers, PyTorch

Error Correction Model	T5-small / BERT-GEC fine-tuned
Completion Model	GPT-2 / GPT-Neo fine-tuned on chat data
API Framework	FastAPI
Database	PostgreSQL (user data), Redis (caching)
Deployment	Docker, Kubernetes
Monitoring	Prometheus + Grafana
Version Control	Git / GitHub

5. Development Guidelines

5.1 Code Standards

- Follow PEP 8 style guidelines for all Python code.
- Write docstrings for every module, class, and function using Google-style format.
- Maintain a minimum of 80% unit test coverage across all modules.
- Use type hints throughout the codebase for improved readability and IDE support.
- Run linting (flake8) and formatting (black) checks before every commit.

5.2 Version Control & Branching Strategy

- Use Git Flow: maintain main, develop, feature/*, bugfix/*, and release/* branches.
- All features must be developed in dedicated feature branches and submitted via Pull Request.
- PR reviews require approval from at least one senior developer before merging.
- Commit messages must follow Conventional Commits format (e.g., feat:, fix:, docs:).
- Tag all releases using Semantic Versioning (MAJOR.MINOR.PATCH).

5.3 Model Development Guidelines

- All models must be trained on properly licensed, bias-audited datasets.
- Maintain experiment tracking using MLflow or Weights & Biases for every training run.
- Document hyperparameters, dataset splits, and evaluation metrics for reproducibility.
- Evaluate models on both benchmark datasets and domain-specific held-out test sets.
- Implement guardrails to prevent the system from generating harmful or offensive content.

5.4 API Design Guidelines

- All endpoints must be versioned (e.g., /api/v1/correct, /api/v1/complete).
- Follow RESTful design principles and return standardized JSON response envelopes.
- Implement OAuth 2.0 / API key authentication for all production endpoints.
- Document all endpoints using OpenAPI 3.0 (Swagger), kept up-to-date with every release.
- Apply rate limiting per user/API key to prevent abuse.

5.5 Testing Strategy

- Unit Tests: Test individual functions and classes in isolation using pytest.
- Integration Tests: Validate module interactions (e.g., preprocessor → correction engine).
- End-to-End Tests: Simulate full user requests through the API layer.
- Performance Tests: Use Locust or k6 to ensure latency SLAs are met under load.
- Regression Tests: Run the full suite on every PR to prevent breaking changes.

5.6 Security Guidelines

- Never log raw user input in production environments — apply data masking.
- Sanitize all inputs before passing to models to prevent prompt injection attacks.
- Store all credentials and secrets in environment variables or a secrets manager (e.g., HashiCorp Vault).
- Conduct quarterly security audits and dependency vulnerability scans (e.g., Dependabot).

6. Evaluation Metrics

The system's performance will be continuously measured using the following metrics:

Metric	Description	Target
GLEU / M2 Score	Grammar error correction quality	≥ 0.60
BLEU Score	Text completion fluency	≥ 0.45
Precision / Recall	Error detection accuracy	$\geq 90\%$
Latency (P95)	API response time at 95th percentile	$< 200\text{ms}$
User Acceptance Rate	% of suggestions accepted by users	$\geq 70\%$
F0.5 Score	Weighted F-score for GEC tasks	≥ 0.65

7. Project Phases & Milestones

Phase 1 — Research & Data Collection (Weeks 1–3)

Literature review, dataset identification and licensing, baseline model benchmarking, and system architecture finalization.

Phase 2 — Core Model Development (Weeks 4–9)

Fine-tune error correction and text completion models. Build preprocessing pipeline and context manager. Establish evaluation harness.

Phase 3 — System Integration (Weeks 10–13)

Integrate all modules into a unified pipeline. Develop and test the FastAPI service. Implement feedback and personalization layer.

Phase 4 — Testing & Optimization (Weeks 14–16)

Conduct full test suite execution, performance profiling, latency optimization, and security audit.

Phase 5 — Deployment & Monitoring (Weeks 17–18)

Deploy to staging and production environments. Configure monitoring dashboards, alerting, and logging pipelines.

Phase 6 — Iteration & Improvement (Ongoing)

Collect user feedback, retrain models on new data, expand language support, and ship feature improvements.

Note: This document serves as the primary reference for all team members. It should be reviewed at the start of each project phase and updated to reflect decisions made during development. All deviations from stated guidelines must be documented and approved by the project lead.